

The Importance of Task Division in Software Development and AI Systems Engineering

Arthur Delfino Duarte

Bachelor's Degree in Electrical Engineering, with emphasis on Electronics and Telecommunications.

arthur.d.duarte@live.com

Abstract—The division of tasks is a fundamental principle in software engineering and, more recently, in artificial intelligence systems based on large language models (LLMs). This article discusses the cognitive limitations of the human brain, the costs of context switching, and how specialization improves productivity and quality. It draws a parallel between traditional software teams and multi-agent AI architectures, demonstrating that dividing complex problems into smaller, specialized tasks produces more reliable, maintainable, and scalable systems.

Index Terms—Multi-agent systems, LLM, software engineering, task division, cognitive load, context switching.

I. INTRODUCTION

There is a reason why virtually all technology companies in the world work with specialized teams rather than expecting a single person to do all the work. At first glance, it may seem that a single developer has better control over the project, as they know all parts of the system. However, as complexity grows, this strategy becomes increasingly less efficient.

Interestingly, the exact same phenomenon occurs when using language models (LLMs). Many users try to build a single giant prompt to do everything, when in reality, modern AI systems are much more efficient when they divide responsibilities among specialized agents. The principle is exactly the same: dividing complex problems into smaller ones produces faster, more reliable, and easier-to-maintain systems.

This article explores this intersection between software engineering and artificial intelligence, presenting theoretical foundations on cognitive load, specialization, and multi-agent architectures, with practical application examples.

II. FUNDAMENTALS OF TASK DIVISION

A. The Biological Limitation of the Human Brain

The first important concept is understanding that no one can hold an entire project in working memory. During

development, there are dozens of simultaneous activities: architecture, database, APIs, frontend, authentication, security, UX, documentation, testing, deployment, monitoring, and optimization. Each of these areas requires a different type of reasoning.

This difficulty does not stem from lack of skill, but from cognitive limits. The human working memory can only maintain a limited amount of information simultaneously. Research in software engineering productivity shows that developers consider a productive day precisely one in which they can work long periods without significant interruptions. In a study with 379 professional developers, interruptions and frequent context switches appeared as one of the main factors perceived as harmful to productivity.

Complexity does not grow linearly; it grows with the interactions between components. In a small system, a developer can keep practically all the context in their head. As new features emerge, not only the amount of code increases, but also the number of relationships between modules, dependencies, and architectural decisions.

B. The Three Fundamental Roles

Even when only one person develops a system, they typically assume three distinct roles. The first is the Planner, responsible for answering what will be built, how it will be built, which architecture to use, and which technologies to employ. Their focus is strategic — they do not write code, they think. The second is the Executor, who transforms planning into code. Their goal is to produce, and they should not be redesigning the architecture while implementing. The third is the Supervisor, who continuously asks: is it working? Are there bugs? Is the code clean? Is it secure? Is it documented? Have the requirements been met? They act as a reviewer.

When a single person does everything, they continuously switch between these three mental modes. This generates cognitive

fatigue and reduces the quality of the work produced.

C. The Invisible Cost of Context Switching

Suppose a developer is implementing authentication. During this task, they mentally maintain: login flow, tokens, database, encryption, sessions, APIs, and client requirements. Suddenly a bug appears in another module. Now they need to unload all this context and load a completely different one. After fixing the bug, they need to reconstruct the previous mental state.

Each context switch requires the brain to mentally reconstruct the previous state. This costs time — a lot of time. Studies in software engineering show that fully recovering context after an interruption can take several minutes, and frequent interruptions significantly reduce productivity. Researchers in human-computer interaction have studied this phenomenon for decades. The results show that frequent interruptions can increase stress, elevate mental load, make it difficult to resume the original task, and reduce the perception of productivity. These effects are especially relevant in creative and analytical activities such as programming, architecture, and code review.

D. The Concept of Cognitive Load

Every brain has limited capacity. It is useful to imagine a backpack: each task adds weight. Architecture, database, frontend, Docker, security, documentation, and testing — each of these areas contributes to the total mental load. When the backpack gets heavy, forgetfulness, bugs, poor decisions, and fatigue arise.

Task division distributes this weight. Instead of a single person carrying everything, each team member carries only their part, resulting in fewer interruptions, less rework, fewer errors, and greater speed. It is not just because there are more people — it is because there is specialization.

III. WORK DIVISION IN SOFTWARE TEAMS

A. How Teams Solve the Problem

Imagine a team where each person has a main responsibility: architect, backend, frontend, QA, DevOps, and documentation. Each professional maintains their focus, and the result is more efficient than if everyone tried to do everything. Work division does not mean just dividing people, but rather dividing responsibilities.

A team of ten people where everyone does everything tends to produce work duplication, architecture conflicts, rework, and inconsistent decisions. Meanwhile, a team where each role is clearly defined — even with the same number of people — tends to be much more efficient because it reduces ambiguities and improves coordination.

B. Production Pipeline

Large companies work like a factory. Requirements go through architecture, which feeds development, followed by testing, deployment, and monitoring. Each stage has clear criteria, which prevents errors from advancing to subsequent phases.

The cost of errors increases exponentially the later they are detected. A bug found during planning costs 1x. During development, 3x. During testing, 10x. After reaching production, 100x. The earlier an error is detected, the lower the cost to fix it — which is why constant reviews exist in well-structured processes.

C. Parallelism

A person working alone needs to perform tasks sequentially: database, then API, then frontend, then testing. Total time is 100%. A team, on the other hand, can work in parallel: database, API, frontend, and testing can occur simultaneously. This is one of the greatest productivity gains provided by work division.

D. Specialization

In the aerospace industry, no one expects a single engineer to design the engine, fuselage, electronics, embedded software, and hydraulic systems alone. Each specialist deepens their knowledge in a specific domain. Software engineering has evolved in the same direction. Specialization reduces individual cognitive effort and increases the quality of decisions within each area.

In 1975, Frederick P. Brooks Jr. published *The Mythical Man-Month*, considered one of the most influential books in software engineering. His most well-known principle, Brooks' Law, states: adding people to a late project tends to make it even later. This is because new members need to learn the architecture, understand business rules, know internal patterns, and communicate with others. Brooks showed that the number of communication channels grows approximately by the formula $n(n-1)/2$, evidencing that coordination also has a cost. This does not mean that large teams are bad,

but that they need organization, clear roles, and well-defined architecture.

A systematic review that analyzed 5,414 initial studies and selected 102 relevant papers found that under deadline pressure, many teams produce more quickly, but software quality tends to decline. This result reinforces that productivity should not be measured only by the amount of code written, but also by the quality and sustainability of the system.

IV. APPLYING THE PRINCIPLE TO ARTIFICIAL INTELLIGENCE

A. The Giant Prompt Problem

Everything we have discussed so far applies almost perfectly to language models. Many users build an enormous prompt trying to make a single model analyze requirements, design architecture, write code, perform testing, document, review security, and optimize performance. In practice, this is equivalent to asking a single professional to simultaneously perform all the roles of a large team.

Imagine a prompt with thousands of lines. The model needs to simultaneously maintain objectives, rules, context, constraints, standards, examples, and architecture. This increases the chance of forgetting requirements, contradicting instructions, mixing responsibilities, and generating inconsistent code. Even with large context windows, reasoning quality can degrade when many responsibilities compete for the model's attention.

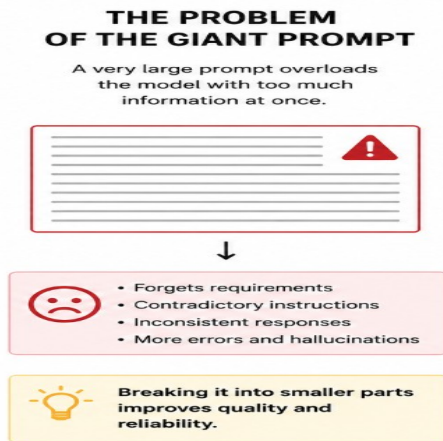


Fig. 1. The giant prompt problem: a single prompt tries to encompass all responsibilities.

B. Agent Engineering

Instead of a single agent responsible for everything, we create several specialized agents, each with an extremely clear function. A typical example includes: Supervisor, Planner, Backend Executor, Frontend Executor, Database Executor, Tester, and Documenter. Each agent receives only the context necessary for its function, which reduces distractions and improves response quality.

Notice that no agent tries to do everything. Each one solves only its specific problem. This is exactly the same principle of work division in human teams.



Fig. 2. Division of complex problems into smaller, specialized problems.

C. Flow Between Agents

The typical flow of a multi-agent system works as follows: the user makes a request to the Supervisor, who forwards it to the Planner. The Planner divides the problem into tasks and distributes them among the specialized executors (Backend, Frontend, Database). Then, a Reviewer checks consistency, the Tester validates quality, and the Documenter prepares documentation. Finally, the consolidated response is delivered to the user.

Each step of the flow is clearly defined, and no agent needs to know all areas of the system in depth. Each one works with a smaller, more focused context.

D. Advantages of Sub-Agents

The multi-agent approach offers several advantages. First, specialization: each agent

receives only the necessary context, which reduces distractions and improves quality. Second, reduced context: instead of a 100-page prompt, each agent receives about 10 pages, increasing quality and reducing hallucinations. Third, reusability: the testing agent can be used in any project, as can the documentation agent — creating reusable components.

Fourth, ease of maintenance: if documentation changes, only the documenter is replaced; the rest remains the same. Fifth and most importantly, parallelism: multiple agents can work at the same time — backend, frontend, database, and documentation can be processed simultaneously, drastically reducing total time.

E. The Orchestrator as Tech Lead

In human teams, there is often someone responsible for coordinating the work: a tech lead, architect, or engineering manager. In AI systems, this role is performed by the orchestrator agent. It interprets the user's request, divides the problem into subtasks, forwards each subtask to the appropriate specialist, integrates the results, and verifies consistency before delivery. The orchestrator does not need to be the best programmer or writer; its main function is to coordinate the workflow.

V. PRACTICAL EXAMPLES AND ANALOGIES

A. Traditional Method vs. Modern Method

In the traditional method, the user sends a request directly to an LLM, which tries to produce the entire system in a single response. This is equivalent to a single developer trying to do everything alone — the result is often inconsistent, superficial, or more error-prone.

In the modern method, an orchestrator receives the request and forwards it to a planner, who generates a task list. These tasks are distributed to specialized executors (API, Database, Frontend). Then, QA validates the result, a documenter prepares the documentation, and the final response is delivered to the user. Much more organized and reliable.

B. Analogy with Civil Construction

Imagine building a building. Would you hire only one person to draw the blueprints, calculate the structure, install the electrical system, do the plumbing, build the walls, paint, review quality, and issue reports? Probably not. There are specialized

professionals: architect, structural engineer, electrician, plumber, mason, painter, and inspector. Each professional masters their area and works in coordination with others.

A multi-agent AI system follows the same logic: an orchestrator agent distributes work to specialists, who then integrate their results. The ideal architecture for complex projects is not a super-agent that does everything, but a team of specialized agents working in a coordinated manner — just like a well-organized software engineering team.

C. Example in Log Analysis Systems

In an environment with Ollama, OpenCode, and local models, a multi-agent architecture could be structured with an orchestrator coordinating a classifier and a planner. The classifier feeds a log reader, while the planner coordinates a Linux specialist. The results flow to error detection and event correlation, which feed a report generator, go through a technical reviewer, and produce the final response.

Each agent can use a short, objective, and highly specialized prompt, reducing context load and facilitating maintenance. This paradigm tends to become increasingly important as LLM-based systems evolve in scale and complexity.

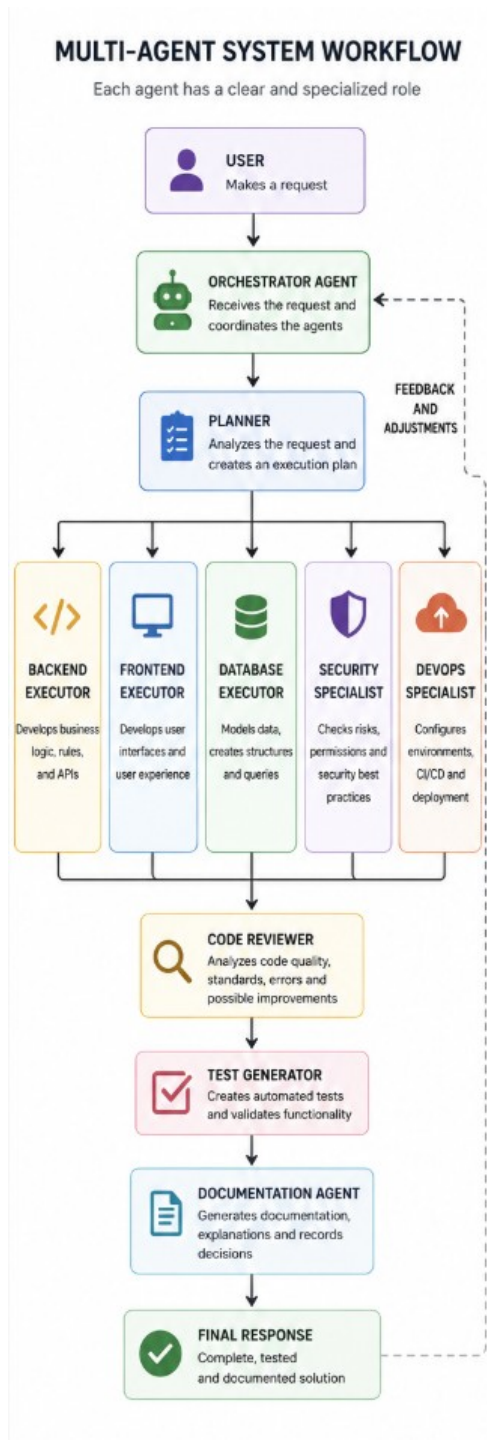


Fig. 3. Workflow in a multi-agent system with orchestrator and specialists.

VI. CONCLUSION

Evidence from software engineering, cognitive psychology, and computer science points in the same direction: complex systems become more efficient when work is decomposed into clear responsibilities. This applies to people and, increasingly, to artificial intelligence.

Instead of concentrating all responsibility on a single developer or a single prompt, modern architectures distribute functions among specialists coordinated by an orchestrator. This approach reduces cognitive load, improves decision quality, reduces error propagation, facilitates testing and maintenance, and enables continuous system evolution.

In the context of LLMs, this paradigm is particularly powerful. An ecosystem of specialized agents can solve complex problems in a more predictable, auditable, and scalable way than a single super-prompt. This change represents a transition similar to what occurred in software engineering: from the figure of the solitary programmer to teams organized by functions, processes, and collaboration based on specialization.

The evolution of software engineering has shown that specialization, division of responsibilities, and well-defined processes produce higher quality systems than concentrating all functions in a single person. Modern AI engineering follows exactly the same principle, and this trend will consolidate as LLM-based systems continue to evolve in scale and complexity.

REFERENCES

- F. P. Brooks Jr. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.
- WikiBooks. *LaTeX/Mathematics*. URL: en.wikibooks.org/wiki/LaTeX/Mathematics (accessed on 05/15/2025).
- P. Lima, E. Guerra, P. Meirelles, L. Kanashiro, H. Silva, and F. Silveira. "A Metrics-Driven Approach for Quality Improvement in Software Development." *Journal of Systems and Software*, 2017.
- J. Schell. *The Art of Game Design: A Book of Lenses*. 2nd ed. A. K. Peters, Ltd., 2014.
- P. Lima, E. Guerra, M. Nardes, A. Mocci, G. Bavota, and M. Lanza. "An Annotation-based Approach for Detecting Software Defects." In: *Proceedings of the International Conference on Software Engineering*, 2017.
- S. C. Müller and T. Fritz. "Stuck and Frustrated or In Flow and Happy: Sensing Developers' Emotions and Progress." In: *Proceedings of the IEEE/ACM 37th IEEE International Conference on Software Engineering*, 2015.
- M. Czerwinski, E. Horvitz, and S. Wilhite. "A Diary Study of Task Switching and

Interruptions." In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2004.

AUTHOR

Arthur Delfino Duarte holds a Bachelor's degree in Electrical Engineering from the Instituto Nacional de Telecomunicações (Inatel) and has solid experience in IT infrastructure, computer networks, data centers, and telecommunications, having worked at companies such as Huawei, Nokia, Net/Claro, and Surf Telecom.

Throughout his career, he has participated in deployment and operation projects for critical environments, involving Linux and Windows systems, routers, switches, data center infrastructure, monitoring, disaster recovery, and large-scale telecommunications services. Currently, he focuses on deepening his knowledge in Software Engineering, Backend Engineering, DevOps, cloud computing, distributed systems, and software architecture, developing practical projects with Java, Spring Boot, Docker, Kubernetes, REST APIs, relational databases, and locally executed artificial intelligence.

He also holds certifications and training in networking, Linux, cloud computing, and project management, including training from Red Hat, Microsoft, Google, AWS, and professional certifications in Nokia, Juniper, and Microsoft Azure technologies.

His main interests focus on Software Engineering, high-availability infrastructure, system reliability, observability, artificial intelligence applied to software development, locally executed language models (LLMs), infrastructure automation, and scalable architectures for modern applications.